

Hospital Appointment and Patient Visit Management System

ABAP Cloud and RAP Technical Project Report

Course	Applied Computer Science
Module	SAP ABAP
Technology	ABAP Cloud, RAP, CDS, OData V4, Fiori Elements
Team Member	Sharath Chandra Chandrashekhar
Team Member	Piyush Prabhakar
Submission	July 2026
GitHub Repository	https://github.com/SharathChandra-dev/SAP-ABAP

Project statement

A draft-enabled SAP Fiori application for maintaining patients and doctors and scheduling validated hospital appointments through ABAP Cloud RAP.

Technical documentation prepared for assessment and project defense

FRONT MATTER

Authorship, Use, and Evidence Statement

This document was prepared for the Hospital Appointment and Patient Visit Management System project by Sharath Chandra Chandrashekhar and Piyush Prabhakar. It describes the design and implementation completed in the assigned ABAP Cloud development environment.

Repository object names, behavior rules, service definitions, screenshots, and troubleshooting outcomes are based on the project package and the Eclipse/Fiori work recorded during development. Where the report discusses a future enhancement or production recommendation, it is labeled as such and is not presented as completed functionality.

The application uses sample academic data. It is not intended to store real patient information or to satisfy production healthcare compliance requirements without additional security, privacy, operational, and testing work.

Evidence boundary

Verified means the team reported successful activation or observed the behavior in Fiori preview. Recommended means the item remains future work.

FRONT MATTER

Submission Profile and Academic Context

This technical report documents the Hospital Appointment and Patient Visit Management System developed with ABAP Cloud and the ABAP RESTful Application Programming Model. The work was completed in Eclipse with ABAP Development Tools and demonstrated through SAP Fiori Elements previews backed by published OData V4 services.

The project was produced by Sharath Chandra Chandrashekhar and Piyush Prabhakar for Applied Computer Science at SRH University Munich. The report is structured for both academic assessment and project defense: it connects requirements to implementation evidence, explains the design decisions, records the working procedures, and identifies limitations without overstating the result.

The principal deliverables are the RAP repository objects, a working appointment application with draft editing and business validation, supporting patient and doctor maintenance applications, and the documentation needed for presentation and knowledge transfer. All custom objects use the customer namespace and do not modify SAP standard objects.

- Course context: SAP ABAP within Applied Computer Science.
- Development environment: Eclipse IDE with ABAP Development Tools.
- Platform approach: ABAP Cloud, RAP, CDS, OData V4, and Fiori Elements.
- Team: Sharath Chandra Chandrashekhar and Piyush Prabhakar.

FRONT MATTER

Executive Summary

The completed solution manages hospital appointments and the related master data required to schedule them. Users can work with appointments, patients, doctors, and departments through metadata-driven Fiori screens. Appointment records store date, time, reason, priority, status, notes, patient, doctor, department, and audit data.

RAP draft capabilities provide stateful create and edit interactions on top of an OData service. Value helps replace manual UUID entry with guided selection. Determinations enrich appointment records with readable patient, doctor, and department information. Validations prevent dates in the past and reject double booking for the same doctor, date, and time. Additional patient and doctor validation protects duplicate IDs and basic contact-data quality.

The implementation demonstrates the end-to-end RAP stack: transparent tables, root CDS view entities, projection views, behavior definitions, behavior pools, metadata extensions, value-help entities, service definitions, OData V4 UI bindings, publication, and Fiori Elements preview. The result satisfies the central technical, documentation, and defense expectations of the supplied exam framework.

- Primary app: draft-enabled appointment scheduling.
- Supporting apps: patient and doctor maintenance.
- Two or more value helps: patient, doctor, department, priority, and status.
- Core safeguards: required fields, date/time rules, duplicate identifiers, and doctor conflict detection.

FRONT MATTER

Table of Contents

The report follows the lifecycle of the project, beginning with business scope and assessment alignment, then moving through architecture, implementation, user experience, testing, deployment, knowledge transfer, and final reflection. Page-level headings are intentionally explicit so that an examiner can quickly locate the evidence behind a design or implementation claim.

- Use the object inventory on page 49 for a quick repository checklist.
- Use the defense guide on page 44 before the live demonstration.
- Use the knowledge-transfer checklist on page 45 for handover.

Part	Coverage	Pages
A	Front matter, scope, requirements, team, and method	1-9
B	Architecture, data model, CDS, RAP behavior, and draft	10-25
C	Fiori metadata, services, UI, and security	26-37
D	Testing, deployment, demo, maintenance, and reflection	38-47
E	References, object inventory, and appendices	48-50

FRONT MATTER

Terminology and Abbreviations

The project combines SAP platform terminology with application-specific language. This glossary fixes the meaning used throughout the report and avoids ambiguity during the defense. UUID fields are technical keys; IDs such as PatientID and DoctorID are human-readable business identifiers.

Term	Meaning	Use in this project
ADT	ABAP Development Tools	Eclipse tooling used to create and activate repository objects.
CDS	Core Data Services	Semantic data models and projections.
RAP	RESTful Application Programming Model	Transactional business-object architecture.
BDEF	Behavior Definition	CRUD, draft, validation, determination, and authorization contract.
OData V4	Open Data Protocol version 4	Published protocol consumed by Fiori Elements.
Draft	Temporary transactional state	Supports create/edit before final activation.
UUID	Universally unique identifier	Technical key generated by managed numbering.
Value help	Guided field selection	Select patient, doctor, department, priority, or status.

PROJECT DEFINITION

Problem Statement

A hospital appointment process coordinates several kinds of information at once: who the patient is, which doctor will attend, which department owns the appointment, when the consultation occurs, why it is required, and what operational status applies. A spreadsheet or disconnected set of forms makes those relationships easy to mistype and difficult to validate.

The project addresses that problem with a structured transactional application. Master data for patients, doctors, and departments is separated from appointment transactions. The appointment business object references the master records through UUID keys while the user interface displays readable IDs and names. This preserves stable technical relationships without forcing users to understand technical keys.

The central quality requirement is not merely to show data. The system must protect the schedule. It therefore validates required fields, prevents historical dates, rejects conflicting doctor bookings, and provides controlled values for priority and status. Draft support lets a user enter data safely before committing an active record.

- Reduce manual entry of technical keys.
- Prevent invalid dates and double booking.
- Provide one consistent source for appointment information.
- Expose the solution through a generated, metadata-driven Fiori experience.

PROJECT DEFINITION

Objectives and Success Criteria

The implementation objective was to produce a complete ABAP Cloud application flow rather than an isolated database exercise. Success was defined through observable behavior in the Fiori preview and through the activated repository objects that support that behavior.

A successful appointment flow allows a user to create a draft, select valid master data through value help, supply schedule details, receive meaningful validation feedback, and activate the appointment. A successful maintenance flow allows patient and doctor records to be created and validated in their own screens. Technical success also requires activation of CDS entities, behaviors, metadata, service definitions, and OData V4 bindings.

The outcome is judged against correctness, usability, architecture, and explainability. The team must be able to demonstrate not only that the screen works, but also how data moves from the persistence table through CDS and behavior into the published service.

- End-to-end RAP object chain is active and published.
- Appointment Create and Edit use draft behavior.
- At least two filterable fields provide value help.
- Validation errors are understandable and block invalid activation.
- Technical documentation and presentation support a live defense.

PROJECT DEFINITION

Scope and Boundaries

The business scope includes patient, doctor, department, appointment, and visit concepts. The delivered interactive emphasis is appointment scheduling, patient maintenance, and doctor maintenance. Department data participates in appointment selection and display, while the separate department maintenance app was intentionally deferred when the project was finalized under time constraints.

The solution does not claim to be a full hospital information system. It does not include billing, insurance, laboratory integration, pharmacy, inpatient bed management, medical imaging, or clinical decision support. It is an academic RAP application that demonstrates transaction handling and user-centered validation around a well-defined workflow.

The existing source package also contains earlier non-draft artifacts and visit models. The final draft-enabled objects use the newer ZR_SHHMS_ and ZC_SHHMS_ naming convention. The report distinguishes the final objects from historical scaffolding so that maintenance work follows the correct path.

- In scope: appointment scheduling and related master-data selection.
- In scope: draft, value help, determinations, validation, service binding, and Fiori preview.
- Partial scope: visit and department models exist, but are not the main demonstrated maintenance flows.
- Out of scope: clinical records integration, payments, and enterprise launchpad deployment.

ASSESSMENT ALIGNMENT

Mapping to the Exam Framework

The supplied framework emphasizes ABAP Development Tools, RAP, CDS semantic models, behavior definitions, OData services, clean-core design, documentation, and a technical walkthrough. The project addresses those expectations directly through repository objects and the final Fiori previews.

Technical execution is demonstrated by the draft-enabled business objects, validation logic, value helps, metadata extensions, and published OData V4 services. Clean core is supported through custom objects, ABAP Cloud-compatible APIs, and the absence of modifications to SAP standard objects. Documentation and reflection are addressed by this report, while the companion slide deck supports the final defense.

Automated test coverage is the least mature assessment area. The supplied package includes a status helper test scaffold, but the final draft-enabled application was primarily verified through activation and manual Fiori scenarios. This limitation is stated openly and converted into a concrete future-improvement item.

Framework area	Project evidence	Weight
Technical Execution and RAP	CDS, behaviors, draft, validations, services, Fiori preview	35%
Clean Core Architecture	Custom namespace, released model, no standard modification	25%
Documentation and Reflection	Technical report, decisions, limitations, lessons	20%
Collaboration and Defense	Team roles, demo flow, knowledge transfer, slide deck	20%

PROJECT MANAGEMENT

Team Roles and Collaboration

Sharath Chandra Chandrashekhkar and Piyush Prabhakar worked as a two-person development team. The implementation was carried out iteratively in Eclipse: create an object, activate it, inspect the activation feedback, correct signatures or model inconsistencies, and then verify the corresponding user behavior in Fiori preview.

In a small academic team, role boundaries must remain practical. Both members need enough understanding to defend the whole architecture, even when one person focuses more on RAP implementation and the other on testing, data, or documentation. Pair review is especially valuable for behavior definitions because a mismatch among root view fields, draft table fields, and mapping fields can invalidate the service at compile time.

The knowledge-transfer process therefore treats the solution as shared ownership. The repository inventory, activation order, validation rules, and demo script are documented so either teammate can explain and operate the system.

Team member	Primary contribution	Shared responsibility
Sharath Chandra Chandrashekhkar	ADT execution, RAP object assembly, Fiori verification	Architecture explanation and demo
Piyush Prabhakar	Review, data scenarios, documentation support	Validation explanation and demo recovery

ARCHITECTURE

Solution Architecture Overview

The application follows the standard RAP layering model. Persistence tables contain durable hospital data and draft tables contain temporary edit state. Root CDS view entities expose the transactional model. Behavior definitions attach create, update, delete, draft, validation, determination, numbering, locking, authorization, and ETag semantics to that model.

Projection views shape the service-facing contract. Metadata extensions describe list columns, selection fields, object-page facets, labels, text arrangement, value-help behavior, and criticality. Service definitions expose projection entities and helper entities. OData V4 UI bindings publish those definitions so Fiori Elements can generate list-report and object-page experiences.

The architecture deliberately keeps business validation in RAP behavior rather than in UI-specific code. This means the same rules apply to any compliant consumer of the service, not only to the preview screen.

Layer	Main objects	Responsibility
Persistence	ZSH_HMS_* and ZSHHMS_*_D	Active and draft data
Business object	ZR_SHHMS_* plus BDEF	Transactional contract and rules
Projection	ZC_SHHMS_*	Service-facing fields and behavior
UI semantics	Metadata extensions and value helps	Generated Fiori behavior
Service	ZUI_SHHMS_* and bindings	Published OData V4 endpoints

ARCHITECTURE

Domain Model

The domain centers on Appointment. Each appointment references one patient, one doctor, and one department and carries schedule details, clinical reason, operational priority, status, notes, and technical audit information. Patient and doctor are reusable master entities rather than text copied into every appointment.

Technical relationships use 16-byte UUIDs because they are stable and well suited to managed numbering. Human-facing identifiers such as P001, DR001, and D001 remain separate. The UI can display IDs and names while the persisted relationship continues to use a durable UUID key.

A Visit concept extends the model toward post-appointment clinical documentation. In the current delivery, appointment scheduling is the complete draft-enabled flow and Visit remains a model and future integration point rather than the principal demonstrated transaction.

Entity	Key business data	Relationship
Patient	Patient ID, name, birth date, contact, allergy	Selected by appointment
Doctor	Doctor ID, name, specialization, contact, active flag	Selected by appointment
Department	Department ID, name, location/contact	Owns doctor and appointment context
Appointment	Date, time, reason, priority, status, notes	References patient, doctor, department
Visit	Diagnosis, treatment, follow-up, status	Related to an appointment

PERSISTENCE

Appointment Table Design

AppointmentUUID is the technical key. AppointmentID is the readable business identifier used in the Fiori header and list. PatientUUID, DoctorUUID, and DepartmentUUID hold the technical foreign references used for value-help selection and validation.

PatientID, DoctorID, DepartmentID, PatientName, DoctorName, and DepartmentName are maintained by a determination. This denormalization was accepted as a pragmatic academic solution because projection-level text associations caused compatibility errors and draft-persistence requirements demanded exact field alignment. The tradeoff is that display values must be refreshed when the referenced master data changes.

AppointmentDate and AppointmentTime are independent fields, which makes schedule filtering simple. Reason and Notes provide free text. Priority and Status are short code fields backed by value helps and UI criticality.

Field group	Examples	Purpose
Keys	AppointmentUUID, AppointmentID	Technical identity and readable identity
References	PatientUUID, DoctorUUID, DepartmentUUID	Stable master-data links
Display cache	PatientName, DoctorName, DepartmentName	Readable Fiori output
Schedule	AppointmentDate, AppointmentTime	Booking slot
Business	Reason, Priority, Status, Notes	Operational appointment data
Audit	CreatedBy/At, LastChangedBy/At, LocalLastChangedAt	Traceability and concurrency

PERSISTENCE

Patient and Doctor Table Design

The patient table records PatientUUID, PatientID, FirstName, LastName, DateOfBirth, Gender, Phone, Email, Address, CriticalAllergy, and audit fields. The doctor table records DoctorUUID, DoctorID, DoctorName, Specialization, DepartmentUUID where applicable, Phone, Email, IsActive, and audit fields.

Business IDs are validated for uniqueness at application level. Contact fields receive basic syntax checks before draft activation. Name normalization capitalizes the first character of each word so that values entered in lower case are stored in a consistent presentation format.

The design keeps validation proportionate to an academic system. Phone length and numeric content are checked, and email must contain the expected separators. Production systems should additionally use country-aware phone formatting, formal email validation, duplicate-match strategies, and identity governance.

- Patient validation: unique ID, name formatting, phone/email quality, sensible birth date.
- Doctor validation: unique ID, title-case name, 10-digit phone, email pattern, controlled active flag.
- Managed UUID numbering removes manual technical-key entry.
- Audit fields support traceability and ETag handling.

DRAFT

Draft Persistence Tables

Each draft-enabled business object requires a draft table whose fields are compatible with the root CDS entity. The appointment object uses ZSHHMS_APPT_D, the patient object uses ZSHHMS_PATIENT_D, and the doctor object uses ZSHHMS_DOCTOR_D. Each draft table contains the business fields plus the administrative include SYCH_BDL_DRAFT_ADMIN_INC.

During development, adding display fields to the root appointment entity immediately created draft-table errors because the draft persistency did not contain matching fields or contained mismatched data types. The issue was resolved by aligning the active table, draft table, root view, behavior mapping, and projection.

This is an important RAP lesson: draft is not a UI switch applied in isolation. It is a persistent transaction model with strict structural expectations. A field exposed by the draft-enabled root must be considered across all layers.

```
@EndUserText.label: 'Draft Database Table for Appointment'
define table zshhms_appt_d {
  key mandt          : mandt not null;
  key appointmentuuid : sysuuid_x16 not null;
  appointmentid      : abap.char(20);
  patientuuid        : sysuuid_x16;
  doctoruuid         : sysuuid_x16;
  departmentuuid     : sysuuid_x16;
  "%admin"           : include sych_bdl_draft_admin_inc;
}
```


CDS MODELING

Root CDS View Entities

ZR_SHHMS_APPT is the final root CDS view entity for appointment behavior. It selects from ZSH_HMS_APPT and exposes business-friendly element names. Similar root entities exist for Patient and Doctor. Root entities are behavior-enabled and therefore define the transactional shape of their business objects.

The root view contains annotations for access control, metadata extension, object-node identity, end-user labels, audit semantics, and value-help references. Associations to value-help entities connect technical UUID fields to patient, doctor, and department selections.

The project initially used ZI_SH_HMS_* interface views. Those remain useful source references, but the generated draft-enabled final implementation uses the ZR_SHHMS_* root naming. The defense should focus on the objects actually bound to the final OData services.

```
@AccessControl.authorizationCheck: #MANDATORY
@Metadata.allowExtensions: true
@EndUserText.label: 'Hospital Appointment'
define root view entity ZR_SHHMS_APPT
  as select from zsh_hms_appt
{
  key appointment_uuid as AppointmentUUID,
    appointment_id      as AppointmentID,
    patient_uuid        as PatientUUID,
    doctor_uuid         as DoctorUUID,
    appointment_date    as AppointmentDate,
    appointment_time    as AppointmentTime
}
```

CDS MODELING

Projection Views

ZC_SHHMS_APPT is the service-facing projection over ZR_SHHMS_APPT. It uses the transactional_query provider contract so that Fiori Elements can consume transactional behavior. The projection contains the fields required by the list report, object page, value helps, display names, and audit semantics.

Projection annotations give user-friendly labels and direct the UI to value-help entities. Technical UUID elements remain necessary to persist relationships, but the metadata extension can hide the technical appointment UUID and show patient, doctor, and department IDs or names instead.

The final projection intentionally avoids ObjectModel.text.association annotations that were rejected for the projection type in the target system. Persisted display values and explicit UI fields provided a reliable alternative compatible with the draft model.

- Root layer: transaction model and persisted behavior.
- Projection layer: consumption contract and UI-facing field selection.
- Metadata extension: visual arrangement, labels, filters, and criticality.
- Projection behavior: exposes the root operations to the service.

RAP BEHAVIOR

Behavior Definition Structure

The root behavior definition is a managed implementation. RAP supplies the standard persistence handling, while the behavior pool implements project-specific determinations and validations. Strict mode 2 enforces explicit behavior contracts and catches missing declarations early.

The definition connects the entity to its active and draft tables, selects ETag and locking fields, defines authorization, marks managed numbering and read-only audit fields, requires mandatory booking fields, and exposes create, update, delete, and draft actions.

A mapping section aligns CDS element names with database columns. This mapping became especially important after readable display fields were added. Every persisted root element must have a compatible destination in both active and draft persistence.

```
managed implementation in class ZBP_R_SHHMS_APPT unique;
strict ( 2 );
with draft;

define behavior for ZR_SHHMS_APPT alias HospitalAppointment
persistent table ZSH_HMS_APPT
draft table ZSHHMS_APPT_D
etag master LocalLastChangedAt
lock master total etag LastChangedAt
{
    create; update; delete;
    draft action Activate optimized;
    draft action Discard;
    draft action Edit;
    draft action Resume;
}
```

RAP DRAFT

Draft Lifecycle

Draft turns the stateless OData interaction into a user-friendly edit session. Create produces a draft instance, Edit creates a draft from an active instance, Resume reopens an existing draft, Discard removes the draft, Prepare runs pre-activation checks, and Activate transfers valid draft data to active persistence.

The appointment UI uses this lifecycle to show Create, Edit, Discard Draft, and final activation behavior. Mandatory-field indicators guide the user, while validations run before the data becomes active. A failed validation leaves the draft available for correction.

Prepare is the correct location to assign validations that must run during draft activation. A determination declared on modify is triggered by field changes and is not assigned to Prepare as an on-save determination. Matching event semantics avoids behavior-definition syntax errors.

Draft operation	Purpose	User-visible result
Create	Start a new draft	Editable object page
Edit	Copy active data into draft	Safe edit session
Prepare	Run checks before activation	Validation messages
Activate	Commit valid draft	Active appointment
Discard	Remove draft changes	Return to active/list state

BUSINESS LOGIC

Appointment Validation Rules

Validation runs on save for create and update and is assigned to the draft Prepare action. It checks that the required appointment ID, patient, doctor, department, date, time, and reason are present. The behavior definition also marks these elements mandatory so Fiori displays required-field indicators.

The schedule rule rejects a date earlier than the current date. When the appointment uses the current date, the time must be later than the current time. This prevents historical appointments from being activated and produces an actionable instruction to choose a future slot.

The rule implementation reports messages against the relevant Date or Time field and appends the appointment key to the failed response. Fiori therefore keeps the draft and visually anchors the problem to the field that requires correction.

Rule	Condition	Response
Required data	Mandatory value is initial	Field-specific error; activation blocked
Past date	AppointmentDate before current date	Select a current or future date
Past time today	Current date and time not in future	Select a later time
Invalid reference	Patient/doctor/department not selected	Choose a value-help entry

BUSINESS LOGIC

Doctor Double-Booking Validation

The most important business rule prevents two active appointments from using the same doctor, date, and time. The validation searches existing appointment persistence for a matching DoctorUUID, AppointmentDate, and AppointmentTime while excluding the current AppointmentUUID during update.

When a conflict exists, the error message identifies the existing patient or appointment so the scheduler knows why the slot is unavailable. This goes beyond a generic duplicate message and supports immediate recovery: choose a different doctor, date, or time.

A production implementation should also define which status values count as occupying a slot. Cancelled appointments should normally be excluded, and configurable appointment duration would require interval-overlap logic rather than exact time equality. The current rule meets the stated requirement of identical date and time conflicts.

```
SELECT SINGLE FROM zsh_hms_appt
  FIELDS appointment_id, patient_name
  WHERE doctor_uuid      = @appointment-DoctorUUID
        AND appointment_date = @appointment-AppointmentDate
        AND appointment_time = @appointment-AppointmentTime
        AND appointment_uuid <> @appointment-AppointmentUUID
  INTO @DATA(conflict).

IF sy-subrc = 0.
  " Report conflict and fail the draft activation
ENDIF.
```

BUSINESS LOGIC

Display-Name and ID Determination

The appointment screen must not force users to interpret UUIDs. The `setDisplayNames` determination reads the selected UUID fields and queries the master tables for `PatientID`, `DoctorID`, `DepartmentID`, `PatientName`, `DoctorName`, and `DepartmentName`.

The determination updates those fields in local mode. It is triggered when relevant data changes and runs before activation so the active appointment carries a readable snapshot. This solution also stabilizes list output because the projection can expose ordinary scalar fields rather than unsupported text associations.

The tradeoff is controlled duplication. If a doctor changes their name after an appointment is created, the old appointment retains the stored name until the determination is triggered again. For this project the snapshot is acceptable and easy to explain; a production design could expose read-only virtual text through a supported association strategy.

```
MODIFY ENTITIES OF zr_shhms_appt IN LOCAL MODE
ENTITY HospitalAppointment
UPDATE FIELDS ( PatientID DoctorID DepartmentID
                PatientName DoctorName DepartmentName )
WITH VALUE #( ( %tky = appointment-%tky
                PatientID = patient_id
                DoctorID = doctor_id
                DepartmentID = department_id
                PatientName = patient_name
                DoctorName = doctor_name
                DepartmentName = department_name ) ).
```

MASTER DATA LOGIC

Patient Validation and Normalization

The patient business object follows the same draft-enabled managed pattern as appointment. PatientUUID is managed and read-only, while PatientID and core identity/contact fields are entered by the user. Prepare runs validation before activation.

The normalization determination converts FirstName and LastName to title case so values such as rahul verma are stored as Rahul Verma. The validation checks that PatientID is not already used by another active record, validates phone length and numeric content, checks basic email syntax, and rejects a date of birth in the future.

Messages use reported-ZrShhmsPatient with element markers so Fiori points to the exact field. Any error also adds the transaction key to failed-ZrShhmsPatient, which prevents activation but preserves the draft for correction.

- Unique PatientID across active patient records.
- First and last names normalized to initial capital letters.
- Phone restricted to the expected numeric length.
- Email requires a basic valid form.
- DateOfBirth cannot be later than the current date.

MASTER DATA LOGIC

Doctor Validation and Normalization

The doctor business object uses managed draft behavior and provides create, update, delete, Edit, Resume, Discard, Prepare, and Activate. DoctorUUID is generated automatically. DoctorID, DoctorName, Phone, and Email are mandatory.

normalizeDoctor splits the doctor name into words, converts each word to lower case, and capitalizes the first character. It also normalizes the doctor ID where required. validateDoctor checks duplicate DoctorID, exact phone length and digit-only content, a basic email pattern, and the allowed active flag.

An earlier implementation short-dumped during draft activation because the determination event/signature and behavior declaration did not align. The stable version declares the determination on modify and keeps Prepare focused on validation. This illustrates why the BDEF declaration and generated local handler signature must always be treated as one contract.

- Duplicate DoctorID is rejected with a readable message.
- DoctorName is stored with the first letter of each word capitalized.
- Phone must contain exactly 10 digits in the project rule.
- Email must contain both @ and a dot.
- IsActive is restricted to the agreed true/false representation.

VALUE HELP

Value-Help Entities

The appointment form provides guided selection instead of manual UUID entry. ZVH_SH_HMS_PATIENT, ZVH_SH_HMS_DOCTOR, and ZVH_SH_HMS_DEPT supply the patient, doctor, and department choices. Separate value-help entities provide controlled Priority and Status codes.

The appointment root/projection applies Consumption.valueHelpDefinition to the UUID or code field. The service definition exposes the helper entities so the OData metadata can describe the value-help relationship to Fiori Elements.

The requirement called for at least two value helps on filterable fields. The implementation exceeds that minimum with patient, doctor, department, priority, and status selection. The user can see IDs and readable descriptions while the selected technical value is stored correctly.

Value help	Key	Readable fields
Patient	PatientUUID	PatientID, patient name
Doctor	DoctorUUID	DoctorID, doctor name, specialization
Department	DepartmentUUID	DepartmentID, department name
Priority	Priority code	Normal, Low, High, Urgent
Status	Status code	Booked, Confirmed, Checked-In, Completed, Cancelled

FIORI METADATA

Metadata Extension Design

The metadata extension is responsible for the generated screen composition. `UI.headerInfo` uses `AppointmentID` as the title and `Reason` as the description. A `GeneralInfo` facet groups the object-page identification fields under `Appointment Details`.

`UI.lineItem` defines list-report columns for Appointment ID, Patient Name, Doctor Name, Department, Date, Time, Reason, Priority, and Status. `UI.selectionField` exposes concise filters for Appointment ID, Date, and Status. Technical audit fields and `AppointmentUUID` are hidden from the everyday screen.

The design deliberately avoids placing every field in the filter bar or list. The earlier generated metadata produced a crowded interface full of UUID and audit filters. The final metadata focuses on the fields a scheduler uses repeatedly.

```
@UI.headerInfo: {
  typeName: 'Appointment',
  typeNamePlural: 'Appointments',
  title: { type: #STANDARD, value: 'AppointmentID' },
  description: { type: #STANDARD, value: 'Reason' }
}
annotate view ZC_SHHMS_APPT with {
  @UI.lineItem: [ { position: 10, label: 'Appointment ID' } ]
  AppointmentID;
  @UI.selectionField: [ { position: 20 } ]
  AppointmentDate;
}
```

FIORI METADATA

Criticality and Readable Output

Fiori criticality communicates operational importance without custom UI code. The Priority field references a calculated criticality element so that an urgent appointment appears with red/error semantic emphasis, high priority can appear as warning, and normal values remain neutral or positive according to the mapping.

Readable output replaces UUID columns with PatientName, DoctorName, DepartmentName, and the corresponding readable IDs. This was one of the most important usability improvements because the early screens displayed long values such as 9e9e2dbd-49f7-1fe1-9fc3-6dd98009d374.

The technical values remain available in the business object and value-help transaction, but the list page and object page prioritize meaning. The result is a system a receptionist can scan rather than a database key viewer.

- Urgent priority: red/error criticality.
- High priority: warning criticality.
- Readable patient, doctor, and department data replaces UUID output.
- Audit and technical key fields are hidden from routine display.

SERVICE LAYER

Service Definitions

The appointment service definition exposes ZC_SHHMS_APPT as HospitalAppoinment and also exposes priority and status value-help entities. Patient and doctor generated applications use their own service definitions and bindings for maintenance.

A service definition is the named contract of entities available to a consumer. It does not itself start an HTTP endpoint. The service binding associates the definition with OData V4 UI and provides activation, publication, preview, and test capabilities in ADT.

The object name HospitalAppoinment contains the generated spelling used in the implementation. It is retained to preserve compatibility. A future cleanup could introduce consistently spelled external names after checking consumers and regenerating metadata.

```
@EndUserText.label: 'HMS Appointment OData V4 Service'
define service ZUI_SHHMS_APPT_O4
  provider contracts odata_v4_ui {
    expose ZC_SHHMS_APPT      as HospitalAppoinment;
    expose ZVH_SH_HMS_PRIORITY as Priorities;
    expose ZVH_SH_HMS_STATUS  as Statuses;
  }
```

SERVICE LAYER

OData V4 Service Bindings

Each UI service is activated and published in the ADT Service Binding editor. The appointment binding provides the primary Fiori preview. Patient and doctor bindings provide separate maintenance previews because they were generated as independent draft-enabled applications.

Publishing creates the local service endpoint and enables Preview for the selected entity set. If Preview displays a start-authorization error for the service group, the likely causes are an unpublished binding, missing service-group authorization, or a stale preview URL from a different binding.

The project encountered service-group authorization messages when unrelated entity sets were exposed through the wrong generated binding. Keeping each generated maintenance app bound to its own active, published service resolved the immediate issue.

- Activate the binding after changing the service definition.
- Publish the local service endpoint.
- Select the intended entity set before choosing Preview.
- Open a new preview after metadata changes to avoid stale browser metadata.
- Verify the service-group name in authorization errors.

USER EXPERIENCE

Appointment List Report

The appointment list report is the operational entry screen. The filter bar supports editing status, appointment ID, date, and status, while the table shows the human-readable booking summary. Create opens a draft object page and each row navigates to appointment details.

The final target layout includes Appointment ID, Patient Name, Doctor Name, Department, Date, Time, Reason, Priority, and Status. Urgent priority is emphasized through criticality. The list is designed for scanning and schedule comparison rather than for displaying all technical fields.

The screenshot below is an earlier verified preview captured before the final readable-name and draft refinements. The current implementation was subsequently verified by the team with readable names and Create/Edit behavior.

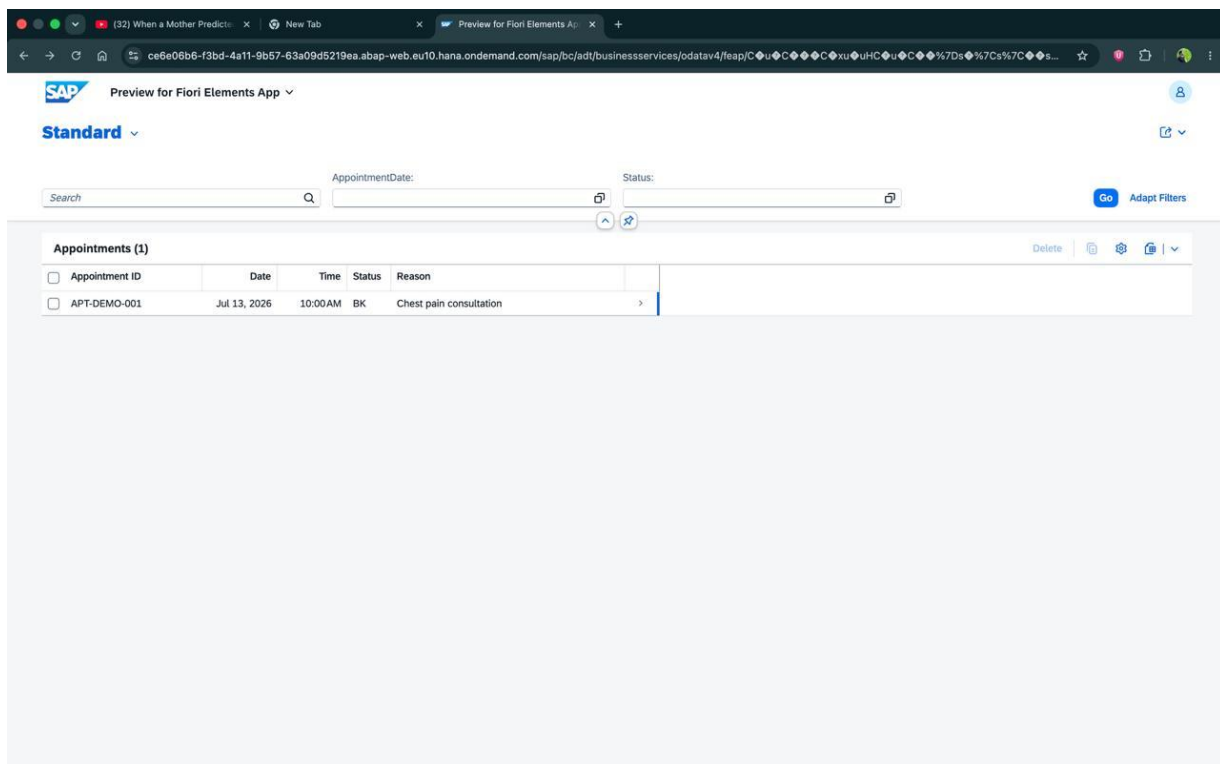


Figure 1. Verified Fiori Elements appointment list preview from the project package.

USER EXPERIENCE

Appointment Object Page

The object page displays the appointment title and reason in the header and organizes the booking fields under Appointment Details. Draft-aware controls show Create or Edit according to the object state, and validation messages appear before activation when data is incomplete or conflicting.

The final object page shows readable patient, doctor, and department IDs/names rather than only technical UUIDs. Priority and status use controlled selection. Audit fields remain hidden in the normal UI while still existing in the service and persistence layers.

The screenshot below records the successful object-page preview used earlier in development. It demonstrates the metadata-driven page structure and the appointment data path from OData to Fiori.

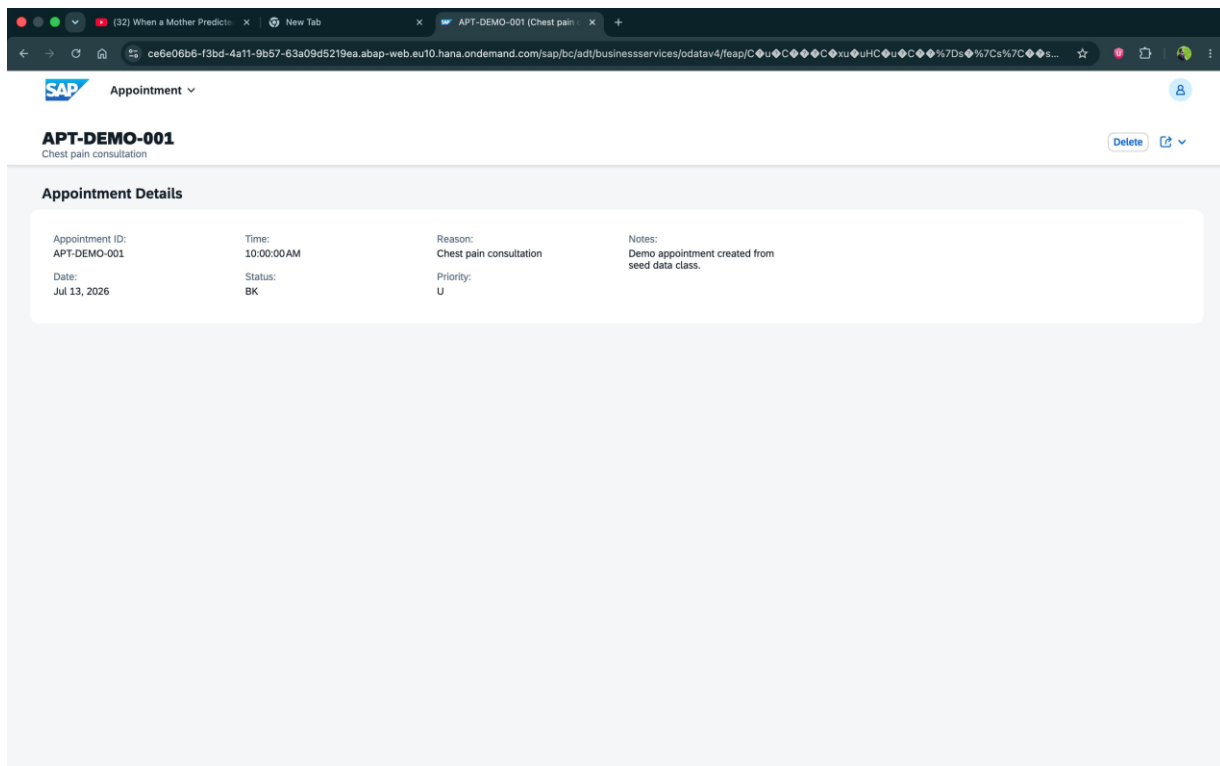


Figure 2. Verified appointment object page with date, time, reason, status, priority, and notes.

USER EXPERIENCE

Patient Maintenance Screen

The patient maintenance service provides its own draft-enabled Fiori list and object page. Users can create and edit patient records without entering a technical UUID. PatientUUID is generated, while PatientID and personal/contact fields are supplied through the form.

The screen validates duplicate PatientID, contact data, and date of birth before activation. First and last names are normalized so the first letter of each word is capitalized. The result is immediately available to the appointment patient value help after service metadata and data are refreshed.

Keeping patient maintenance separate from the appointment list simplified service authorization and allowed each generated root business object to retain its own draft lifecycle. A launchpad or custom shell could later provide one navigation hub for all apps.

- Draft Create and Edit.
- Managed PatientUUID.
- Unique PatientID validation.
- Name normalization and basic contact validation.
- Patient records consumed by appointment value help.

USER EXPERIENCE

Doctor Maintenance Screen

The doctor maintenance service uses a generated draft-enabled root business object. Users enter DoctorID, DoctorName, Specialization, Phone, Email, and the active flag. DepartmentUUID was removed from the visible form according to the final usability decision.

The behavior rejects a duplicate DoctorID and invalid contact values. The determination converts each word in DoctorName to initial-capital format. A failed validation produces a field-specific message and leaves the draft available for repair instead of creating an invalid active doctor.

During implementation, generic RAISE_SHORTDUMP errors were resolved by aligning the behavior definition event with the local handler method and avoiding unsupported determination assignments to Prepare. This troubleshooting path is documented for future maintainers.

- DoctorID is mandatory and unique.
- DoctorName is normalized to title case.
- Phone requires exactly 10 digits in the project rule.
- Email receives a basic syntax check.
- IsActive uses a controlled representation.

SECURITY

Authorization and Access Control

The generated root behaviors declare global authorization and provide `get_global_authorizations` handler hooks. Access-control annotations vary between the earlier and final objects, and the final deployment should align them with the intended course environment and user roles.

The development system produced service-group start-authorization messages when a user previewed a service without the corresponding authorization or used the wrong binding. The operational fix is not to suppress security but to activate/publish the correct binding and ensure the user has permission for that service group.

For academic demonstration, the application runs under the assigned ABAP Cloud development user. A production solution should define business roles such as receptionist, clinician, and administrator and implement instance/global authorization decisions accordingly.

Concern	Current project treatment	Production recommendation
Repository access	ADT project/package authorization	Transport and change-control roles
Service start	Published binding and service-group access	Business catalog and role assignment
Data operations	Authorization handler hooks	Role- and instance-aware decisions
Sensitive data	Academic sample data	Privacy, logging, retention, and least privilege

CLEAN CORE

ABAP Cloud and Clean-Core Alignment

The solution uses custom Z objects and the ABAP Cloud development model. Database tables, CDS entities, behavior definitions, classes, service definitions, and bindings are owned by the project package. No SAP standard object is modified.

RAP provides a released, upgrade-stable extension architecture. Business rules are encapsulated in custom behavior pools, and the Fiori experience is derived from metadata rather than copied or modified SAP UI code. This supports the clean-core objective in the exam framework.

Clean core is an architectural discipline, not simply a naming convention. Future development should continue to use released APIs, avoid classic unreleased function modules, isolate custom persistence, and keep service contracts explicit.

- Custom namespace and package ownership.
- No modification to SAP standard tables, classes, or applications.
- CDS and RAP contracts separate persistence, behavior, and service exposure.
- OData V4 and Fiori Elements minimize custom UI code.
- Released APIs should remain the default for future integration.

QUALITY

Error Handling and Troubleshooting Lessons

Most project defects appeared as activation errors, service compilation errors, or generic Fiori RAISE_SHORTDUMP messages. Activation errors were usually caused by mismatched field names, missing draft-table elements, incompatible types, incorrect behavior method signatures, or an operation declared in the behavior without a matching handler.

Service-binding errors often propagated from a CDS or behavior syntax error. The reliable diagnostic order is to activate the lowest dependency first: table, draft table, root CDS, behavior definition, behavior pool, projection, projection behavior, metadata, service definition, then service binding.

A browser error is the final symptom, not necessarily the root cause. ADT Problems, activation logs, behavior syntax checks, and the dump details should be inspected before repeatedly refreshing the UI.

Symptom	Likely cause	Recovery
Draft persistency unsuitable	Missing or mismatched draft field	Align active table, draft table, CDS, mapping
Behavior syntax error	Declaration/handler mismatch	Regenerate or correct local method signature
Service compile error	Upstream CDS/BDEF inactive	Activate dependency chain bottom-up
Start authorization error	Wrong/unpublished binding or role	Publish correct binding and verify access
RAISE_SHORTDUMP	Runtime handler/model mismatch	Open dump details and simplify event logic

TESTING

Testing Strategy

Testing combines syntax activation, service compilation, manual transaction scenarios, and data verification. Every repository object must activate before the service can be considered valid. Fiori tests then confirm that metadata, draft state, value help, validation, and activation behave as expected from the user perspective.

The project package includes an ABAP Unit scaffold for appointment status helper logic. The final draft-enabled objects were primarily validated through manual end-to-end scenarios. This is sufficient evidence for the academic demonstration but not the target state for a production application.

Future automated tests should isolate the validation methods using RAP test doubles, cover conflict and date boundaries, and verify determination output. Test data should include cancelled appointments, same-day edge cases, duplicate IDs with different case, invalid contact data, and concurrency conflicts.

- Static: syntax check and activation for all dependencies.
- Service: binding compile, publish, metadata, and entity-set preview.
- Functional: create, edit, discard, activate, and delete where permitted.
- Negative: invalid dates, duplicate IDs, conflict slots, invalid phone/email.
- Regression: readable names, value helps, dropdowns, and criticality remain intact.

TESTING

Functional Test Cases and Results

The following matrix records the scenarios confirmed during iterative development. Where the team explicitly reported a working result, the status is Verified. Scenarios identified as recommended are not presented as completed evidence and should be run before a production release.

ID	Scenario	Expected result	Status
T01	Preview appointment list	Existing appointments displayed	Verified
T02	Open appointment detail	Object page displays details	Verified
T03	Create appointment draft	Editable draft opens	Verified
T04	Past date/time	Activation blocked with error	Verified
T05	Doctor slot overlap	Conflict blocked and patient identified	Verified
T06	Patient/doctor/department value help	Readable selections available	Verified
T07	Patient maintenance create	Valid patient activated	Verified
T08	Doctor maintenance create	Valid doctor activated	Verified
T09	Urgent criticality	Urgent shown in red	Verified
T10	High-volume conflict test	No duplicate slot under load	Recommended

DEPLOYMENT

Activation and Publication Order

RAP artifacts form a dependency graph, so activation order matters. Tables and draft tables come first, followed by root CDS entities, behavior definitions, behavior pool classes, projection views, projection behaviors, metadata extensions, value-help entities, service definitions, and service bindings.

When a higher object reports a generic compilation error, activate each lower dependency individually and resolve its own Problem entries. Mass activation is useful only after the dependency chain is structurally consistent.

After service changes, activate and publish the binding, then open a new Fiori preview. Browser refresh alone may retain older OData metadata.

1. Activate active persistence tables and draft tables.
2. Activate value-help and root CDS view entities.
3. Activate root behavior definitions and behavior pools.
4. Activate projection CDS views and projection behaviors.
5. Activate metadata extensions and service definitions.
6. Activate and publish each OData V4 UI service binding.
7. Open a new preview and run the end-to-end smoke tests.

DEPLOYMENT

Run Guide in Eclipse ADT

The application runs from the ABAP system; the local project folder is a source and documentation package rather than an executable desktop program. Eclipse ADT is used to open the ABAP project, navigate the package, edit and activate objects, and preview published services.

For the main demonstration, open the appointment service binding, confirm that the endpoint is Published, select HospitalAppointment, and choose Preview. In the browser select Go to load appointments. Use Create for a new draft or open an existing row and choose Edit.

Patient and doctor maintenance use their own published service bindings. If the preview shows no results, click Go without filters. If it shows a service-group authorization error, return to the exact binding for that generated app and verify activation, publication, and user access.

8. Open Eclipse and connect to ABAP project L02_EN.
9. Open package ZS4D400_100008892_RAP.
10. Open the required Service Binding under Business Services.
11. Confirm Active and Published status.
12. Select the entity set and click Preview.
13. Click Go, then demonstrate Create/Edit and a validation scenario.

DEMONSTRATION

Live Demonstration Script

The live defense should tell one coherent story rather than opening objects at random. Begin with the business problem, show the architecture quickly, demonstrate the main user flow, then return to the behavior code to explain why the rules worked.

Use one positive appointment and one negative appointment. The positive scenario proves value help, draft, readable data, and activation. The negative scenario proves that validation is server-side and prevents a doctor double booking or a past appointment.

Keep the patient and doctor maintenance examples brief. Their purpose is to show that the master data consumed by appointment value helps can also be maintained through RAP draft screens.

14. Introduce the hospital scheduling problem and the two-person team.
15. Show the appointment table, root CDS, BDEF, metadata, and binding as one RAP chain.
16. Open the appointment list and point to readable patient/doctor/department fields.
17. Create a draft, select value helps, and activate a valid future appointment.
18. Attempt a conflicting slot and explain the validation message.
19. Open patient or doctor maintenance and show draft Create.
20. Close with clean-core design, limitations, and next steps.

KNOWLEDGE TRANSFER

Knowledge-Transfer Document and Handover

A knowledge-transfer document should explain how another developer can understand, run, troubleshoot, and extend the solution. In ADT, a command labeled Create New Knowledge Transfer may belong to an installed tool or collaboration extension rather than a RAP repository object. The correct content is still the same: project context, object map, dependencies, runbook, known issues, and ownership.

After opening the New Knowledge Transfer dialog, enter a concise title such as Hospital Appointment RAP - Technical Handover. Select the project/package if the dialog asks for scope, include Sharath Chandra Chandrashekhara and Piyush Prabhakar as owners, and use this report or the dedicated KT section as the source description. Do not select the mistaken flight objects.

If the dialog offers automatic context collection, include the final appointment, patient, and doctor service objects and exclude generated flight examples. Save or generate the transfer, review the collected object list, then share the generated link/file with the teammate or professor according to the tool's workflow.

- Title: Hospital Appointment RAP - Technical Handover.
- Scope: package ZS4D400_100008892_RAP and final ZR/ZC/ZBP/ZUI objects.
- Owners: Sharath Chandra Chandrashekhara and Piyush Prabhakar.
- Include: architecture, activation order, bindings, validation rules, test cases, and known issues.
- Exclude: Z8892FLIGHT and other accidental practice objects.

REFLECTION

Key Technical Challenges

The project encountered realistic RAP integration issues. The first was understanding that ABAP objects live in the SAP repository rather than as executable files in the local Eclipse project folder. The second was coordinating generated object names with hand-built earlier objects.

Draft introduced strict field alignment across active persistence, draft persistence, CDS, and mapping. Text associations that looked attractive for showing names were not allowed in the target projection, so the design moved to persisted display fields populated by determination. Behavior handlers also required exact event categories and generated signatures.

Finally, service preview errors demonstrated that UI symptoms can come from upstream model compilation or authorization. The team learned to diagnose from the lowest dependency and to treat the service binding as the final integration point, not as the origin of every error.

- Repository object lifecycle in ADT.
- Draft persistency compatibility.
- Projection annotation limitations in the target release.
- Behavior event/signature consistency.
- Service-group publication and authorization.

REFLECTION

Design Decisions and Tradeoffs

The system uses separate generated patient and doctor applications instead of a single custom navigation shell. This delivered reliable draft maintenance within the available time and remained consistent with RAP/Fiori Elements generation. A launchpad-style hub remains a future integration step.

Readable patient, doctor, and department values are persisted on the appointment. This adds controlled duplication but avoids unsupported projection text associations and ensures draft-table compatibility. The determination provides a clear, testable synchronization mechanism.

Validation uses simple exact date/time overlap and basic contact syntax. These rules directly satisfy the academic requirements and are easy to defend. Production scheduling would need appointment durations, cancelled-status exclusions, locale-aware contact validation, and stronger authorization.

Decision	Benefit	Tradeoff
Separate maintenance apps	Fast, reliable generated draft flows	No single launchpad navigation shell
Persist display names/IDs	Readable stable list output	Potential refresh when master data changes
Exact slot conflict	Clear requirement coverage	No duration-based overlap
Basic contact checks	Useful data-quality guard	Not internationally comprehensive

ROADMAP

Future Enhancements

The most valuable next enhancement is a unified Fiori launchpad or application shell that presents Appointments, Patients, Doctors, Departments, and Visits as navigable tiles or routes while preserving their separate RAP services. This directly addresses the original request for one entry point without forcing unrelated roots into one transactional business object.

Scheduling can be expanded with duration, working hours, doctor availability, room/resource allocation, recurring appointments, waitlists, reminders, and cancellation reasons. Visit can become a child composition or a linked follow-up workflow after the appointment reaches Checked-In or Completed.

Quality improvements include comprehensive ABAP Unit and RAP integration tests, set-based determinations, formal authorization roles, privacy controls, and deployment to a managed launchpad rather than relying on ADT preview.

- Unified launchpad navigation across all hospital apps.
- Department and Visit maintenance with draft.
- Appointment duration and interval-overlap validation.
- Doctor working hours and availability calendars.
- Notifications, reminders, and waitlist support.
- Automated tests, role-based authorization, and production operations.

CONCLUSION

Conclusion and Learning Outcomes

The project delivers a functioning ABAP Cloud RAP solution for hospital appointment scheduling with draft editing, value helps, readable master data, business validation, OData V4 exposure, and metadata-driven Fiori Elements screens. Patient and doctor maintenance complete the core master-data loop required by appointment creation.

The strongest learning outcome is understanding RAP as a connected contract. Persistence, CDS, behavior, draft, projection, metadata, service definition, binding, and UI must agree. The project also demonstrates that many user-experience improvements can be achieved through semantic modeling and metadata rather than custom frontend code.

The result is appropriate for an academic defense because it includes both successful functionality and documented engineering judgment. Known limitations are explicit, and the object inventory and runbook make the work transferable to another developer.

- End-to-end RAP architecture implemented and understood.
- Draft and validation used to protect transactional quality.
- Value helps and metadata improved usability.
- Clean-core custom development maintained.
- Troubleshooting and knowledge-transfer practices documented.

REFERENCES

References

The report uses the supplied project package, the project conversation record, the SRH assessment framework, and official SAP Help documentation. SAP sources were selected for current technical definitions of RAP, draft, service bindings, ABAP Cloud, and released APIs.

- SAP Help Portal. ABAP RESTful Application Programming Model. <https://help.sap.com/docs/abap-cloud/abap-rap/abap-restful-application-programming-model>
- SAP Help Portal. Draft. <https://help.sap.com/docs/abap-cloud/abap-rap/draft>
- SAP Help Portal. Draft Business Object. <https://help.sap.com/docs/abap-cloud/abap-rap/draft-business-object>
- SAP Help Portal. Using the Service Binding Editor for OData V4. <https://help.sap.com/docs/abap-cloud/abap-development-tools-user-guide/using-service-binding-editor-for-odata-v4-service>
- SAP Help Portal. ABAP Cloud. <https://help.sap.com/docs/abap-cloud>
- SAP Help Portal. Design with Programming Model Aspects. <https://help.sap.com/docs/abap-cloud/abap-cloud/design-with-programming-model-aspects>
- SAP Help Portal. Released APIs. https://help.sap.com/docs/ABAP_Cloud/abap-development-tools-user-guide/released-apis
- SRH University project framework. `sap_abap_cloud_exam_framework.docx`.
- Hospital Appointment RAP project package and Eclipse activation/preview evidence, July 2026.

APPENDIX

Final Object Inventory

This inventory identifies the principal final objects to show during the defense and include in a knowledge transfer. Earlier ZI_SH_HMS_* objects can remain in the package as development history, but the final draft-enabled transaction flow uses the ZR_SHHMS_* and ZC_SHHMS_* object family.

Area	Final objects	Purpose
Appointment	ZSH_HMS_APPT, ZSHHMS_APPT_D, ZR_SHHMS_APPT, ZC_SHHMS_APPT, ZBP_R_SHHMS_APPT	Draft appointment BO
Patient	ZSH_HMS_PATIENT, ZSHHMS_PATIENT_D, ZR_SHHMS_PATIENT, ZC_SHHMS_PATIENT, ZBP_R_SHHMS_PATIENT	Draft patient BO
Doctor	ZSH_HMS_DOCTOR, ZSHHMS_DOCTOR_D, ZR_SHHMS_DOCTOR, ZC_SHHMS_DOCTOR, ZBP_R_SHHMS_DOCTOR	Draft doctor BO
Value helps	ZVH_SH_HMS_PATIENT, ZVH_SH_HMS_DOCTOR, ZVH_SH_HMS_DEPT, ZVH_SH_HMS_PRIORITY, ZVH_SH_HMS_STATUS	Guided selection and code lists
Services	ZUI_SHHMS_APPT_O4 plus appointment binding; patient and doctor service definitions/bindings	Published OData V4 UI
Historical	ZI_SH_HMS_*, ZC_SH_HMS_*, ZUI_SH_HMS_O4	Earlier non-draft scaffolding/reference

APPENDIX

Defense Questions and Model Answers

Why RAP? RAP provides a structured ABAP Cloud model for transactional services. In this project it connects CDS data modeling, managed behavior, draft, validation, OData V4, and Fiori Elements without a custom frontend.

Why UUIDs and readable IDs? UUIDs are stable technical keys used for relationships and managed numbering. PatientID, DoctorID, DepartmentID, and AppointmentID are readable business identifiers. The UI displays the readable values while persistence keeps UUID relationships.

What is draft? Draft stores an incomplete or edited transaction separately until Prepare and Activate succeed. It protects active data and gives the user a safe edit session.

Where is validation executed? In the RAP behavior pool on save and during draft Prepare. The UI receives reported messages, but the business rule is enforced on the service side.

How is clean core supported? All objects are custom, no SAP standard object is modified, and the solution uses ABAP Cloud/RAP extension mechanisms and service contracts.

What would you improve? Add automated RAP tests, duration-aware scheduling, role-based authorization, Visit/Department maintenance, and a launchpad that navigates all apps from one entry point.

- Demonstrate one valid draft activation and one rejected conflict.
- Explain the object chain from table to Fiori in under two minutes.
- State limitations directly; do not claim production readiness.
- Keep the service bindings published before the defense.

Project Repository

GitHub Repository: <https://github.com/SharathChandra-dev/SAP-ABAP>